



Future Makers Summer Camp 2026

July 7, 2026



Canadian University Dubai

School of Engineering, Applied Science and Technology (SEAST)

Prepared by: **Tahamid Hossain**

Supervisor: **Dr. Haythem El-Messiry**

Workshop day one



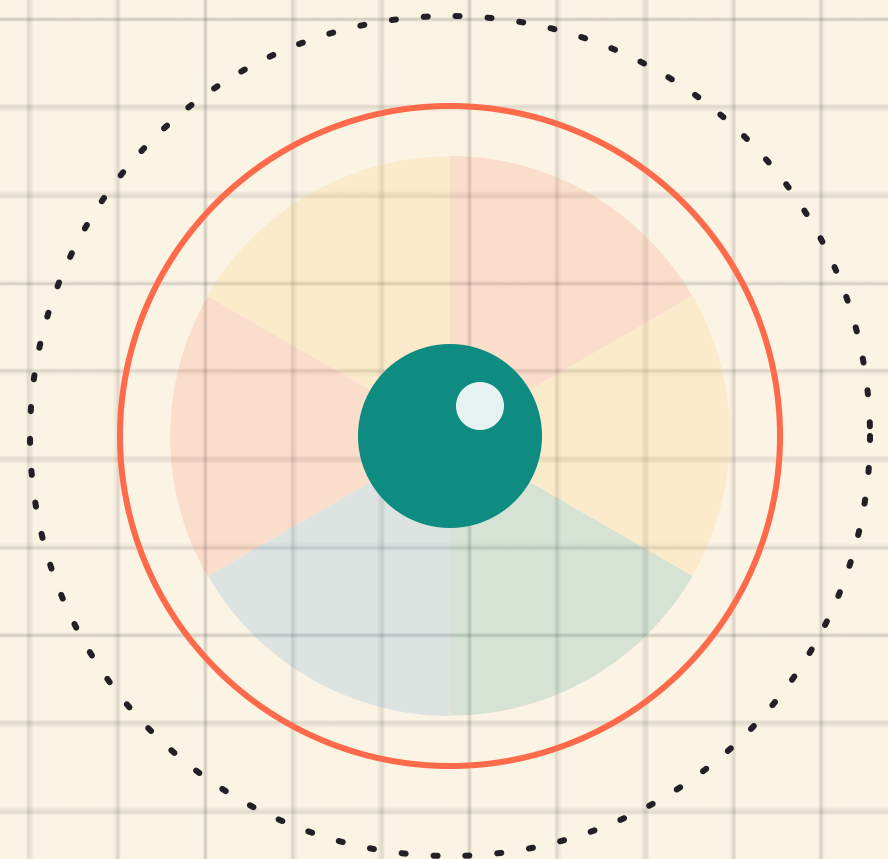
Teach a Computer to See

Every image is secretly just numbers. We'll open that box together, then build and train **a tiny neural network** that reads your handwritten digits.

Pixels & filters

Neural nets

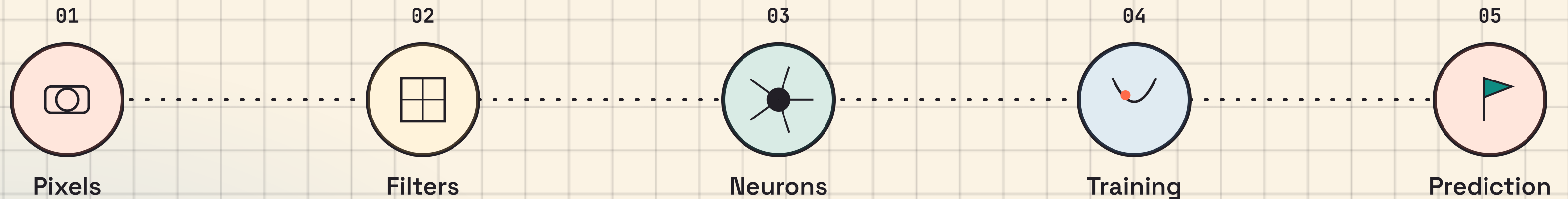
Live demo



Where we're headed

What We'll Build, **Step by Step**

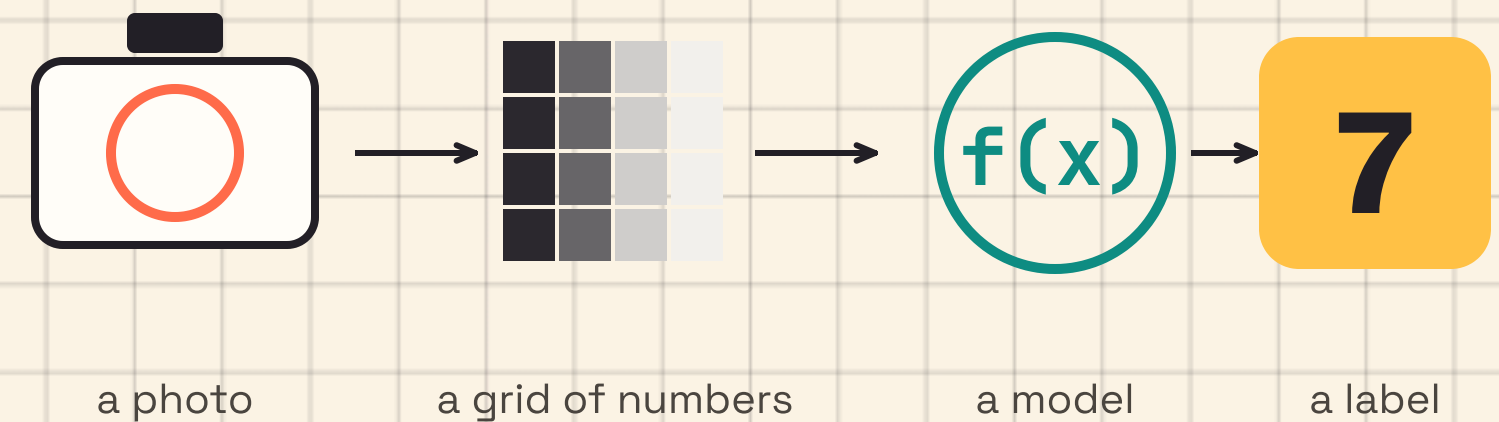
By the end of today you'll understand every gear inside a real image-recognition model
— and you'll have trained one yourself.



Computer Vision: Teaching Software to Understand Images

It's the field that lets computers turn raw pixels into meaning — spotting faces, reading street signs, or recognizing a handwritten 7.

camera → pixels → math → a label a human understands



Every Image Is Secretly Just Numbers

A grayscale image is a grid of pixels — each one a brightness value from **0** (black) to **255** (white).

read_pixels.py

```
image.shape      # (28, 28)  - 28 rows, 28 columns
image[10][14]    # 231      - brightness of one pixel
image[3][3]      # 12       - almost black
```



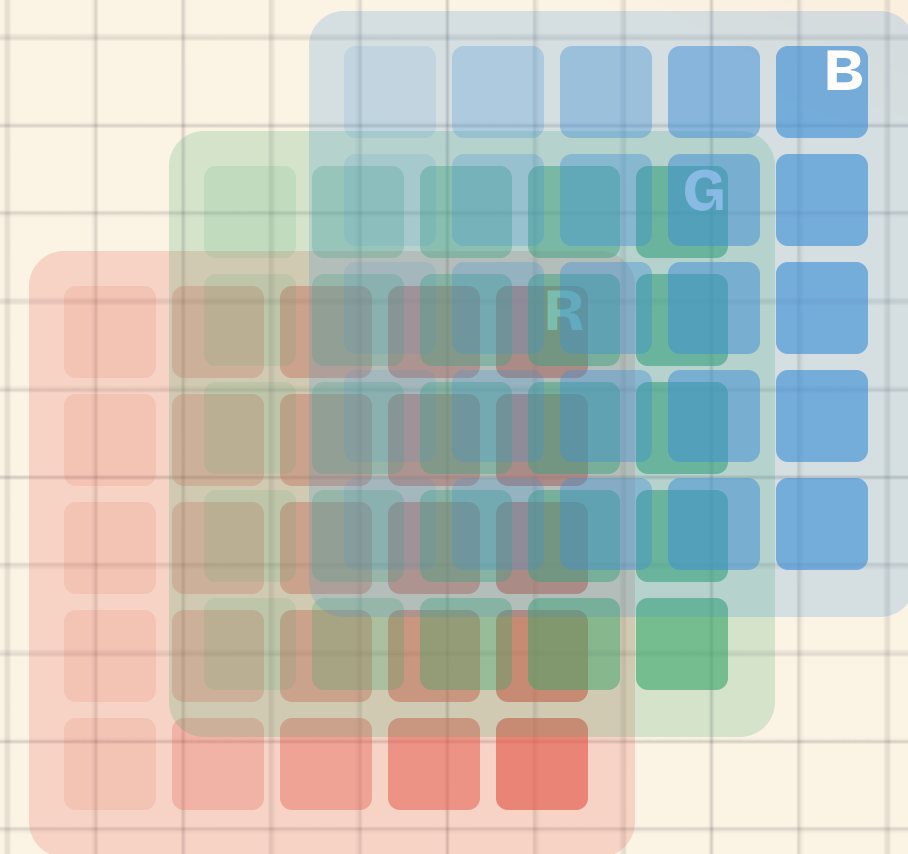
a zoomed-in corner of a digit, as numbers

Color = Three Grids Stacked Together

Color images add Red, Green, and Blue channels — one grid per color, layered like transparent sheets of glass.

color_channels.py

```
image.shape      # (28, 28, 3)
image[10][14]    # [231, 84, 12] — R, G, B
image[10][14][0] # 231           — just the red value
```

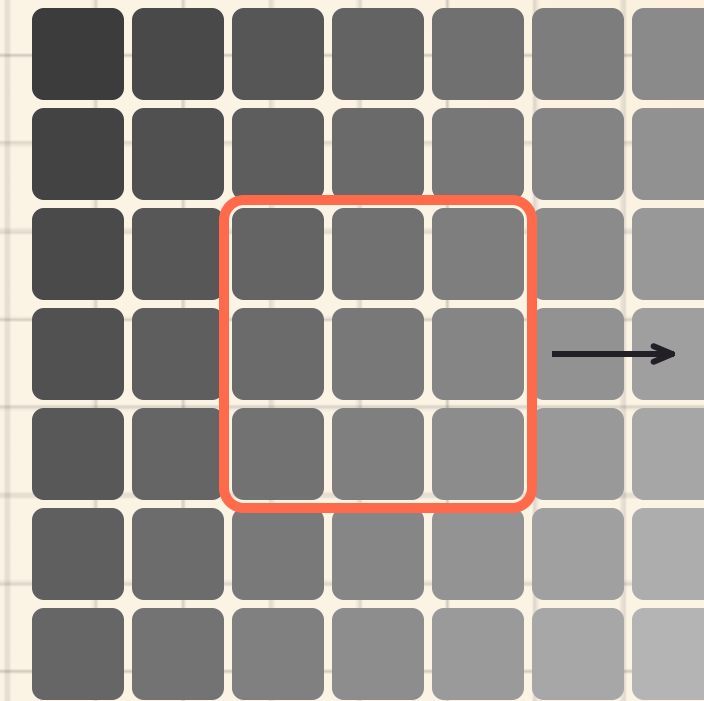


Filters Slide a **Tiny Window** Across the Image

A kernel is a small grid of numbers that slides over the image, multiplying and adding as it goes.

convolve.py

```
def convolve(patch, kernel):  
    total = 0  
    for r in range(3):  
        for c in range(3):  
            total += patch[r][c] * kernel[r][c]  
    return total
```



-1	0	+1
-2	0	+2
-1	0	+1

the kernel (3×3)

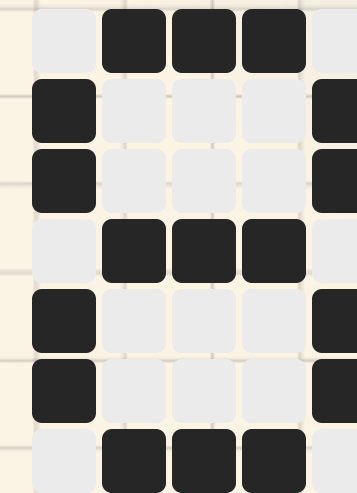
the 3×3 window slides one step at a time

Different Filters Reveal Different Features

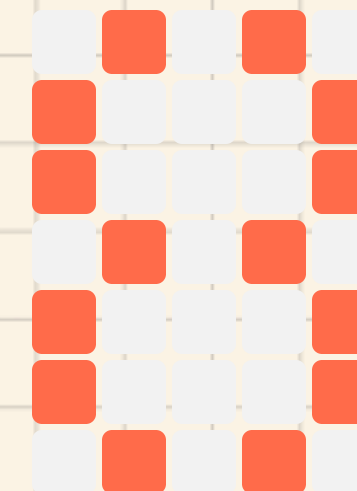
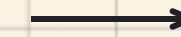
An edge-detecting filter lights up wherever brightness changes sharply — the outline of the digit.

edge_kernel.py

```
edge_kernel = [  
    [-1, 0, 1],  
    [-2, 0, 2],  
    [-1, 0, 1],  
] # lights up vertical edges
```



before: raw grayscale



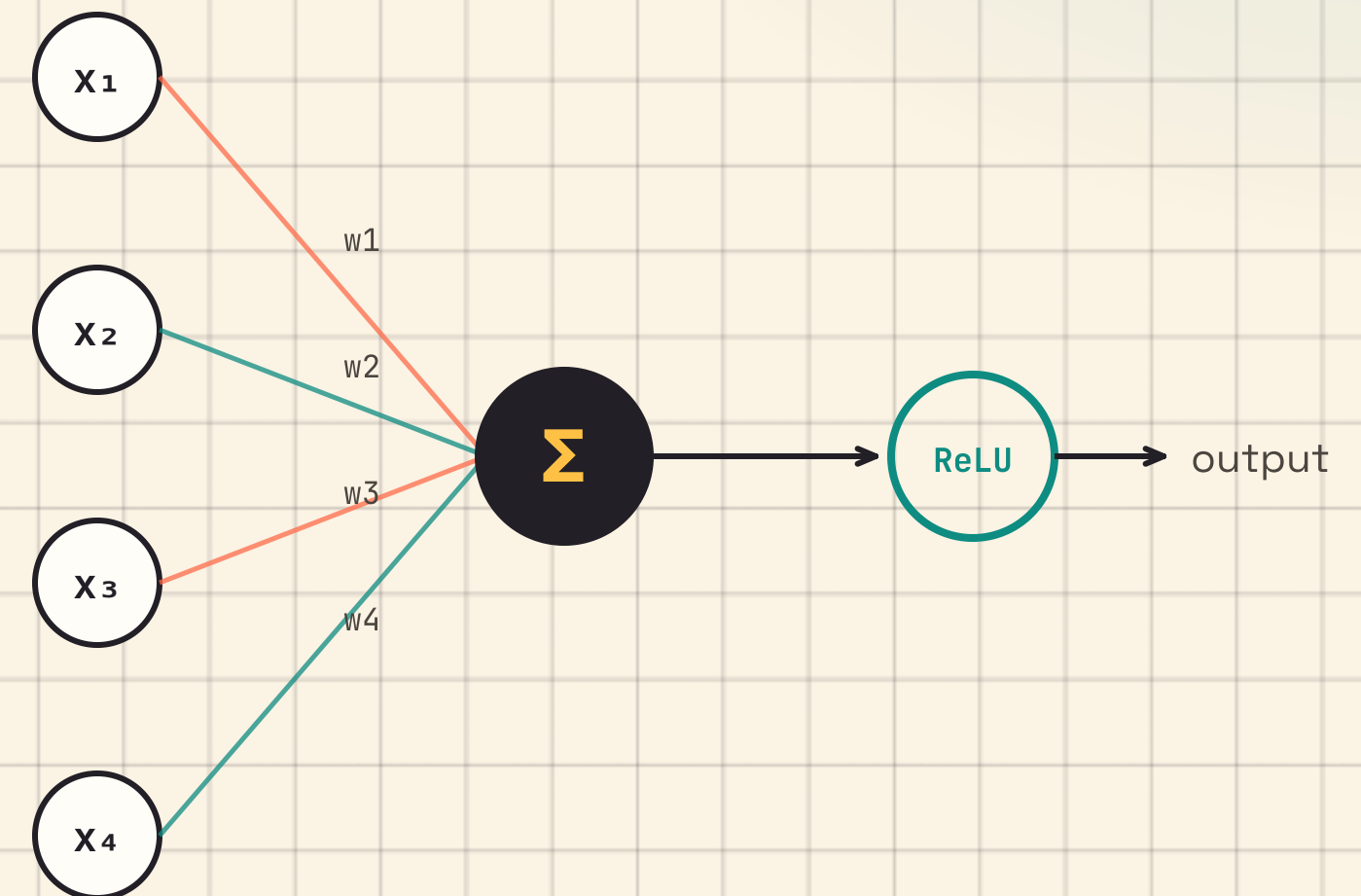
after: edges highlighted

A Neuron **Weights Its Evidence**

Each neuron multiplies every input by a weight, adds them up, and asks: is this enough to fire?

neuron.py

```
def neuron(inputs, weights, bias):  
    total = sum(i * w for i, w in zip(inputs, weights))  
    return total + bias
```



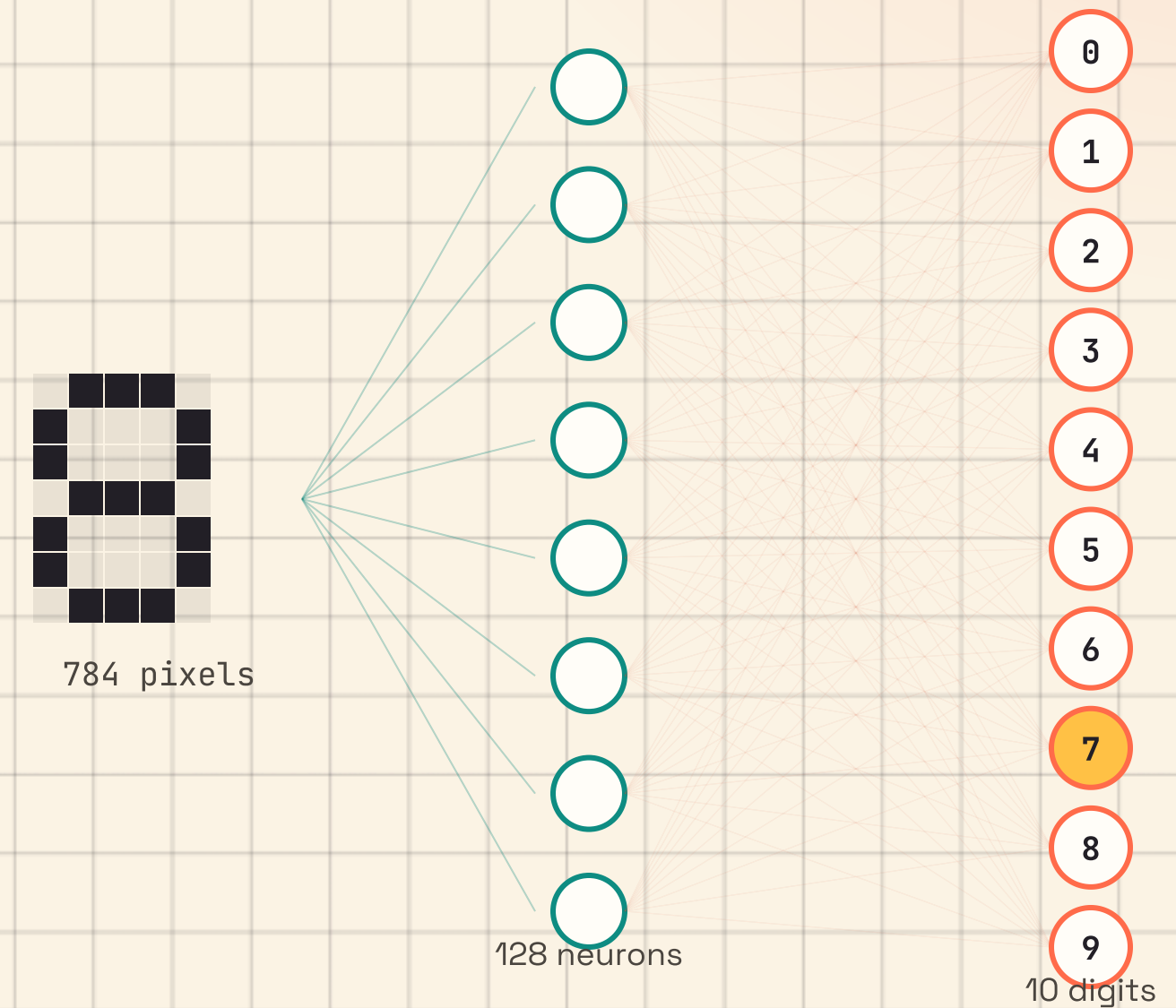
04 · neurons

Stack Thousands of Neurons Into **Layers**

Our network takes **784** pixel values in, thinks with a hidden layer, and outputs **10** digit scores.

layers.py

```
W1 = random_matrix(784, 128) # input → hidden  
W2 = random_matrix(128, 10) # hidden → output
```

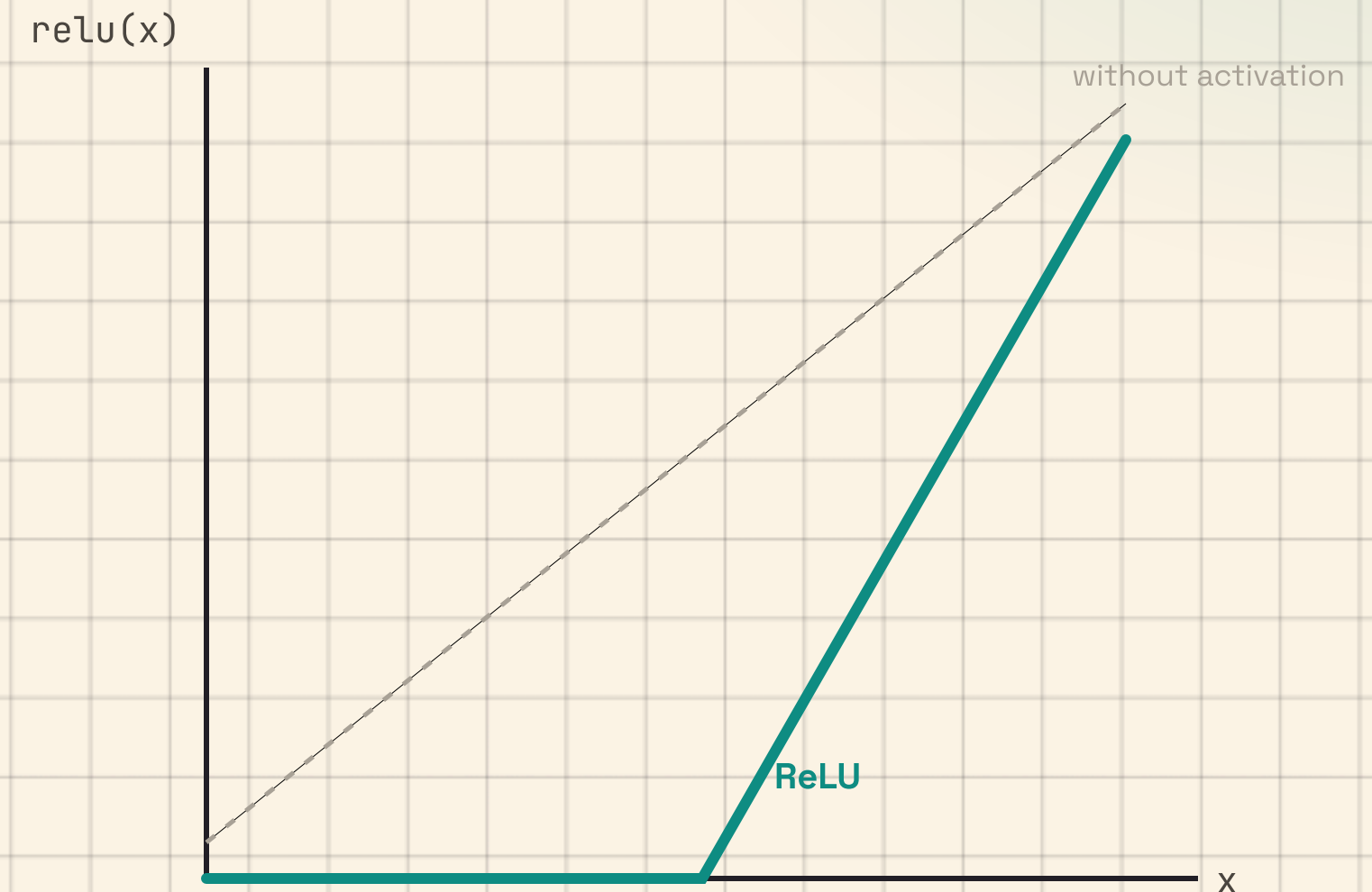


Activation Functions Let the Network **Bend**

Without them, stacking layers is pointless. **ReLU** keeps positive signals and zeroes out the rest.

relu.py

```
def relu(x):  
    return max(0, x)
```



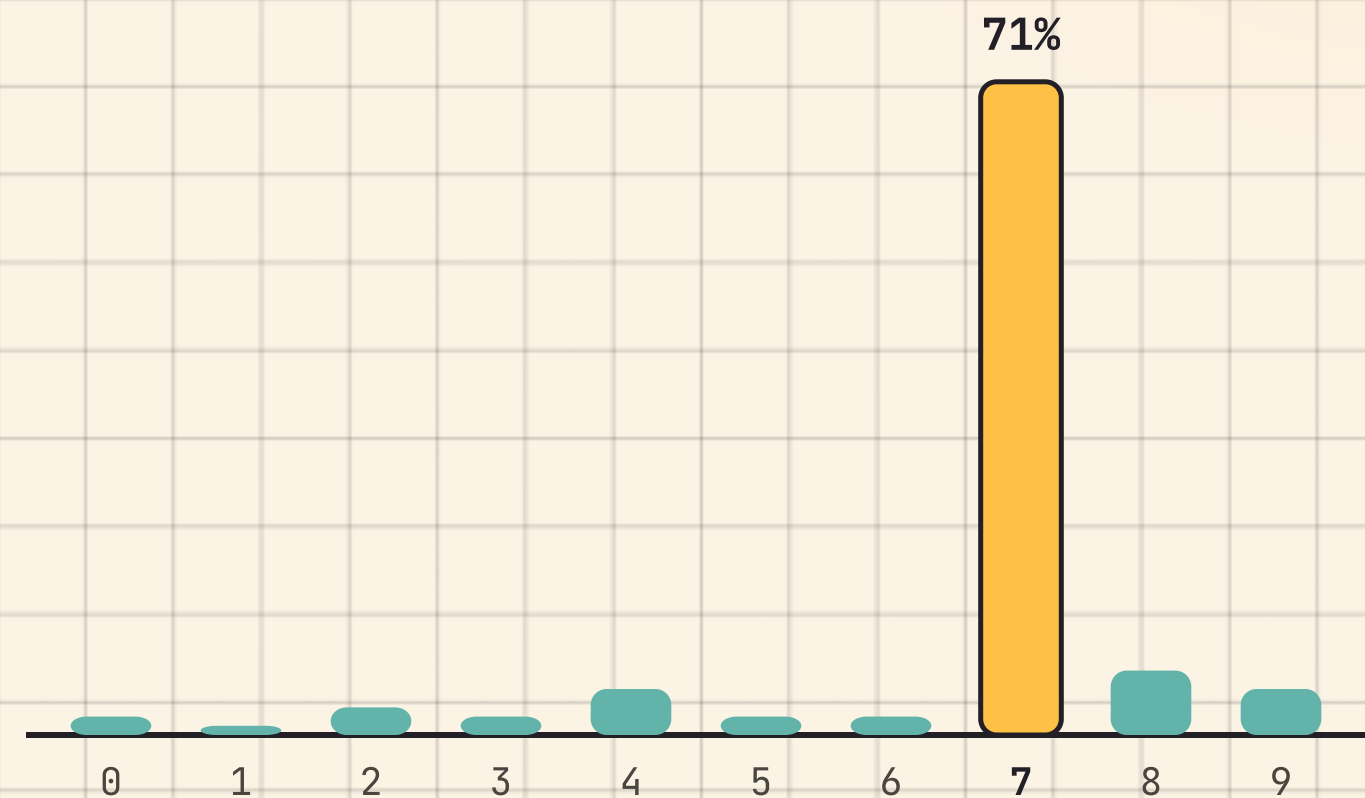
Softmax Turns Scores Into Confidence

The output layer's raw numbers become a confidence percentage for each digit, 0 through 9.

softmax.py

```
import math
```

```
def softmax(scores):  
    exps = [math.exp(s) for s in scores]  
    return [e / sum(exps) for e in exps]
```



Loss Measures **How Wrong** the Guess Was

We compare the network's confidence to the true answer — a big gap means a big loss.

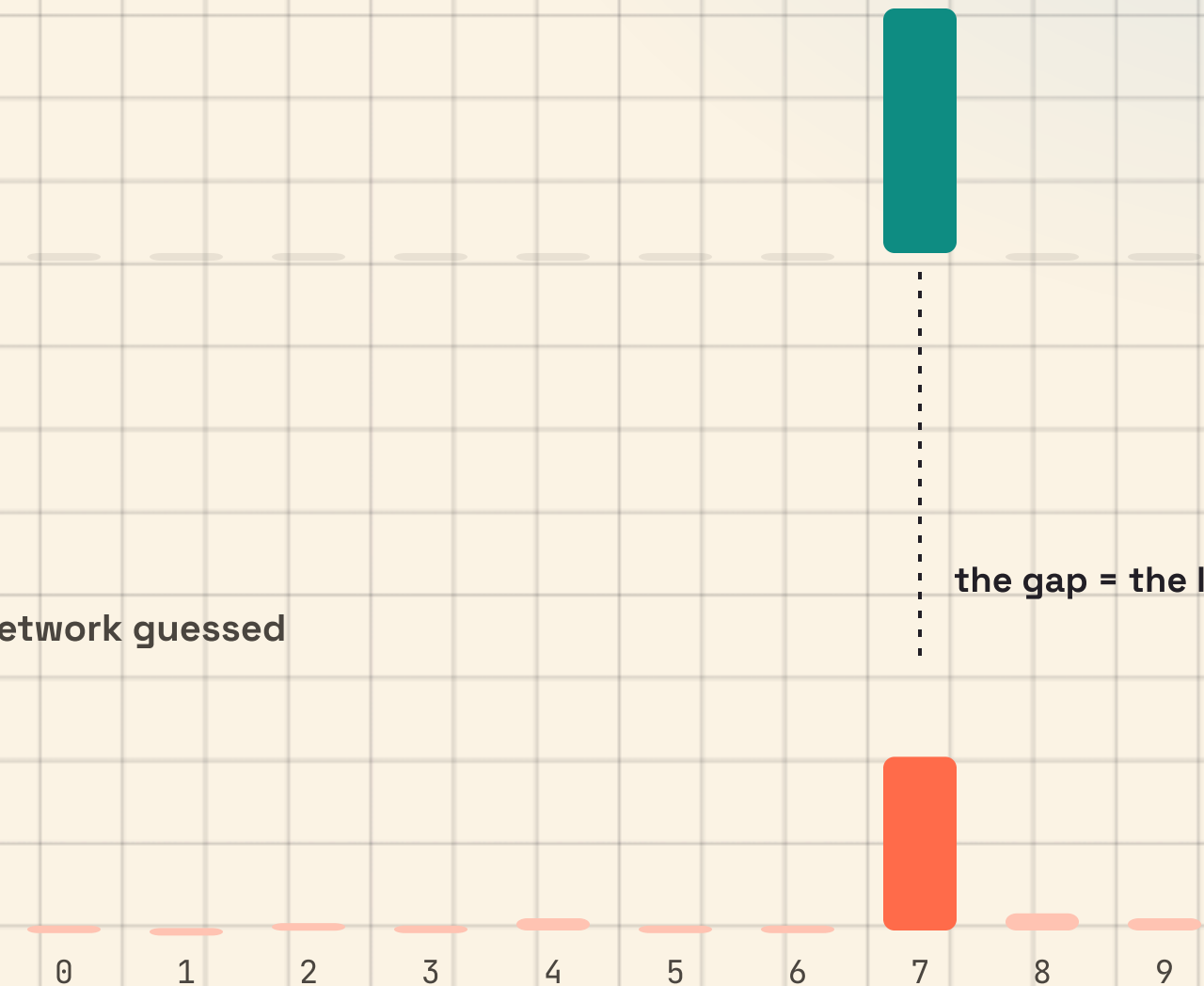
loss.py

```
def loss(predicted, true_label):  
    return -math.log(predicted[true_label])
```

true answer: 7

network guessed

the gap = the loss



Gradient Descent **Rolls the Loss Downhill**

After every mistake, we nudge each weight slightly in the direction that reduces the loss.

update.py

```
weight = weight - learning_rate * gradient
```



Time to Build It Yourself

You've seen every piece — pixels, filters, neurons, training. Now let's put them together, train a real digit-recognizing network, and test it on your own handwriting.

Open the exercise →

Deep Dive

